

Locating Type Errors *Speedify* with Delta Debugging

Joanna

Sharrad

University of Kent
jks31@kent.ac.uk

Olaf Chitil

University of Kent
oc@kent.ac.uk

Example

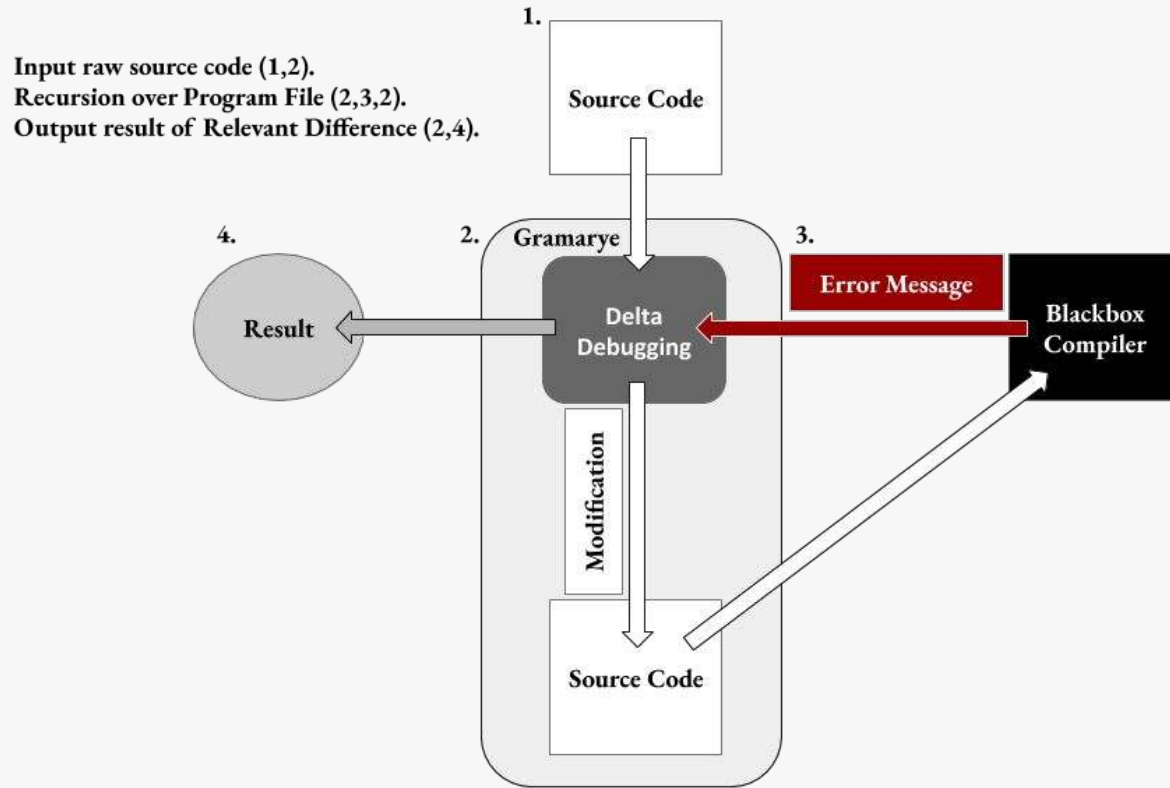
Ill-typed case statement:

```
1 | f x = case x of
2 |   0 -> [0]
3 |   1 -> 1
```

```
1 | f x = case x of
```

Non type-variable argument in the
constraint: Num [a2]

Example from : Chen and Erwig. 2014. Counter-factual typing for debugging type errors.



Sharrad, J., Chitil, O., Wang, M. 2018. Delta Debugging Type Errors with a Blackbox Compiler.

Example

This code has a type error.

```
1 | f x = case x of  
2 |   0 -> [0]  
3 |   1 -> 1
```

Example from : Chen and Erwig. 2014. Counter-factual typing for debugging type errors.

Example

Applying Delta Debugging:

```
1| f x = case x of
2|   0 -> [0]
3|   1 -> 1
```

```
1|
2|
3|
```

Example

Applying Delta Debugging:

```
1| f x = case x of  
2|   0 -> [0]  
3|
```

PASS (Compiles)

```
1|  
2|  
3|   1 -> 1
```

UNRESOLVED

Application of the Moiety Algorithm

Pre-processing to avoid line-splits causing unresolves

```
1| f x = case x of
2|   0 -> [0]
3|   1 -> 1
4| plus :: Int -> Int -> Int
5| plus = (+)
6| fib x = case x of
7|   0 -> f x
8|   1 -> f x
9|   n -> fib (n-1) `plus` fib (n-2)
```

Application of the Moiety Algorithm

1	
2	$\emptyset \rightarrow [\emptyset]$
3	
4	
5	
6	
7	
8	
9	

parse error on input

Application of the Moiety Algorithm

1	
2	
3	1 -> 1
4	
5	
6	
7	
8	
9	

parse error on input

Application of the Moiety Algorithm

```
1|  
2|  
3|  
4| plus :: Int -> Int -> Int  
5|  
6|  
7|  
8|  
9|
```

not parse error on input

(3,4)

Application of the Moiety Algorithm

```
1|  
2|  
3|  
4|  
5| plus = (+)  
6|  
7|  
8|  
9|
```

not parse error on input

(3,4) (4,5)

Application of the Moiety Algorithm

```
1|  
2|  
3|  
4|  
5|  
6| fib x = case x of  
7|  
8|  
9|
```

not parse error on input

(3,4) (4,5) (5,6)

Application of the Moiety Algorithm

```
1|  
2|  
3|  
4|  
5|  
6|  
7| 0 -> f x  
8|  
9|
```

parse error on input

(3,4) (4,5) (5,6)

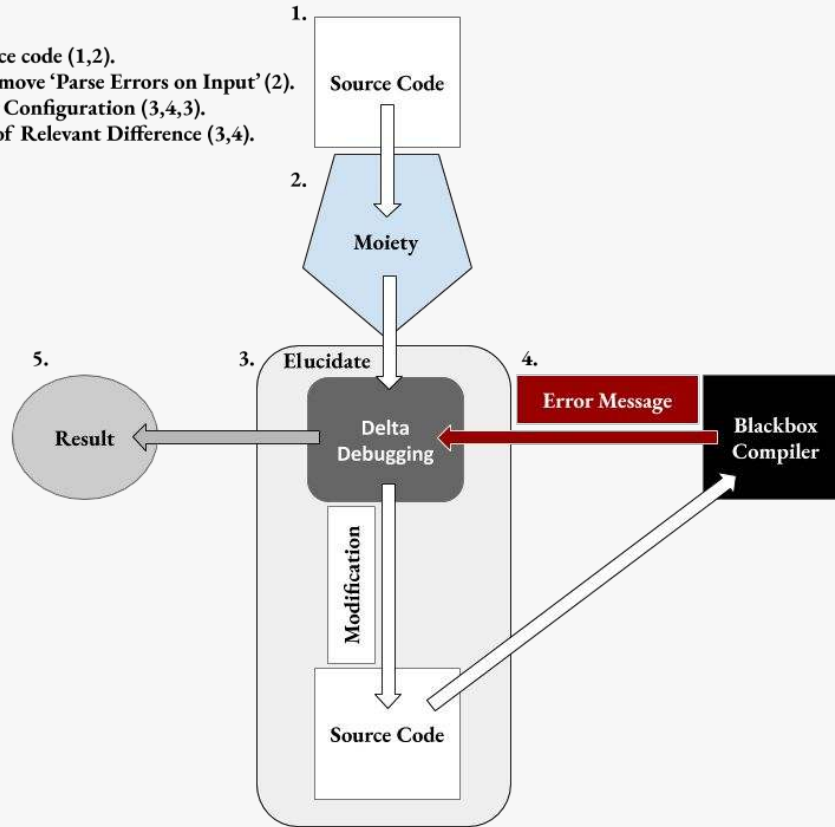
Application of the Moiety Algorithm

```
1| f x = case x of
2|   0 -> [0]
3|   1 -> 1
4| plus :: Int -> Int -> Int
5| plus = (+)
6| fib x = case x of
7|   0 -> f x
8|   1 -> f x
9|   n -> fib (n-1) `plus` fib (n-2)
```

Final Moieties (splitting points):

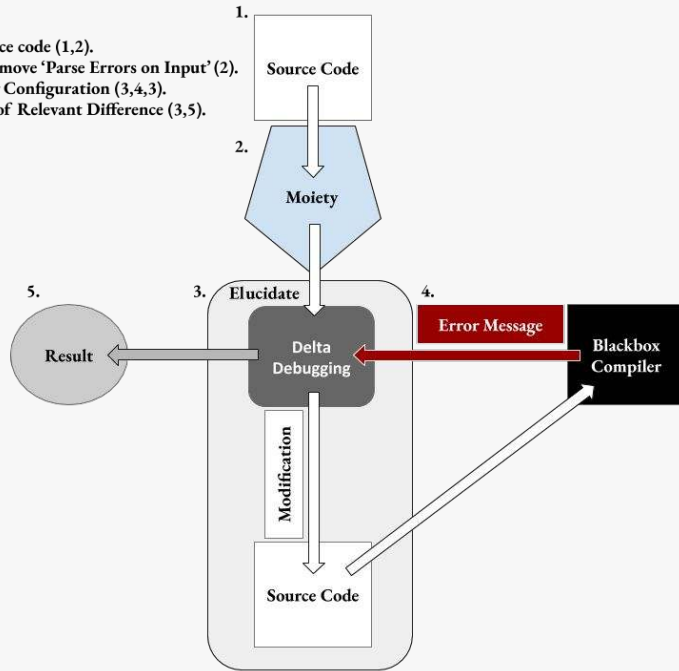
(3,4) (4,5) (5,6)

Input raw source code (1,2).
Pre-check to remove 'Parse Errors on Input' (2).
Recursion over Configuration (3,4,3).
Output result of Relevant Difference (3,4).

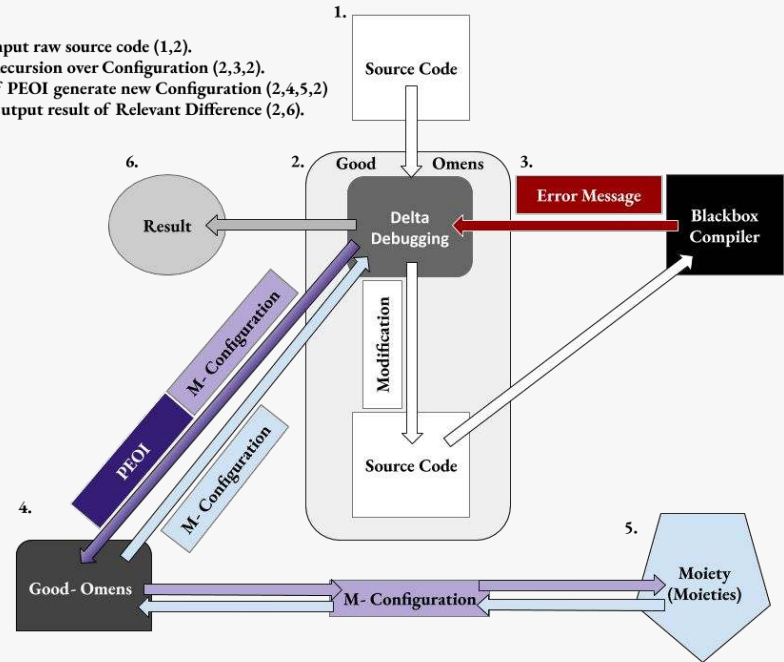


Sharrad and Chitil. 2020. Scaling Up Delta Debugging of Type Errors

Input raw source code (1,2).
 Pre-check to remove 'Parse Errors on Input' (2).
 Recursion over Configuration (3,4,3).
 Output result of Relevant Difference (3,5).



Input raw source code (1,2).
 Recursion over Configuration (2,3,2).
 If PEOI generate new Configuration (2,4,5,2)
 Output result of Relevant Difference (2,6).



Applying the Good-Omens Algorithm

On request 'parse error on input' removal

```
1| f x = case x of
2|   0 -> [0]
3|   1 -> 1
4| plus :: Int -> Int -> Int
5| plus = (+)
6| fib x = case x of
7|   0 -> f x
8|   1 -> f x
9|   n -> fib (n-1) `plus` fib (n-2)
```

```
1|
2|
3|
4|
5|
6|
7|
8|
9|
```

Applying the Good-Omens Algorithm

On request 'parse error on input' removal

```
1| f x = case x of
2|   0 -> [0]
3|   1 -> 1
4| plus :: Int -> Int -> Int
5| plus = (+)
6|
7|
8|
9|
```

FAIL (Type Error)

```
1|
2|
3|
4|
5|
6| fib x = case x of
7|   0 -> f x
8|   1 -> f x
9|   n -> fib (n-1) `plus` fib (n-2)
```

UNRESOLVED

Applying the Good-Omens Algorithm

On request 'parse error on input' removal

```
1| f x = case x of
2|   0 -> [0]
3|   1 -> 1
4|
5|
6|
7|
8|
9|
```

FAIL (Type Error)

```
1|
2|
3|
4| plus :: Int -> Int -> Int
5| plus = (+)
6|
7|
8|
9|
```

PASS (Compiles)

Applying the Good-Omens Algorithm

On request 'parse error on input' removal

```
1| f x = case x of
2|   0 -> [0]
3|
4|
5|
6|
7|
8|
9|
```

PASS (Compiles)

```
1|
2|
3|   1 -> 1
4| plus :: Int -> Int -> Int
5| plus = (+)
6|
7|
8|
9|
```

Parse Error on Input

Applying the Good-Omens Algorithm

On request 'parse error on input' removal

```
1| f x = case x of
2|   0 -> [0]
3|
4|
5|
6|
7|
8|
9|
```

PASS (Compiles)

```
1|
2|
3|   1 -> 1
4| plus :: Int -> Int -> Int
5| plus = (+)
6|
7|
8|
9|
```

Parse Error on Input

Applying the Good-Omens Algorithm

On request 'parse error on input' removal

Configuration Splitting Points:

{[1],[2],[3],[4],[5]}

```
1|  
2|  
3|    1 -> 1  
4| plus :: Int -> Int -> Int  
5| plus = (+)  
6|  
7|  
8|  
9|
```

Parse Error on Input

Applying the Good-Omens Algorithm

On request 'parse error on input' removal

Configuration Splitting Points:

{[1],[2],[3],[4],[5]}

```
1|  
2|    0 -> [0]  
3|    1 -> 1  
4| plus :: Int -> Int -> Int  
5| plus = (+)  
6|  
7|  
8|  
9|
```

Applying the Good-Omens Algorithm

On request 'parse error on input' removal

Configuration Splitting Points:

{[1],[2,3],[4],[5]}

```
1|  
2|    0 -> [0]  
3|    1 -> 1  
4| plus :: Int -> Int -> Int  
5| plus = (+)  
6|  
7|  
8|  
9|
```

Parse Error on Input

Applying the Good-Omens Algorithm

On request 'parse error on input' removal

Configuration Splitting Points:

{[1],[2,3],[4],[5]}

```
1| f x = case x of
2|   0 -> [0]
3|   1 -> 1
4| plus :: Int -> Int -> Int
5| plus = (+)
6|
7|
8|
9|
```

Applying the Good-Omens Algorithm

On request 'parse error on input' removal

Configuration Splitting Points:

{[1,2,3],[4],[5]}

```
1| f x = case x of
2|   0 -> [0]
3|   1 -> 1
4| plus :: Int -> Int -> Int
5| plus = (+)
6|
7|
8|
9|
```

FAIL (Type Error)

Applying the Good-Omens Algorithm

On request 'parse error on input' removal

```
1| f x = case x of
2|   0 -> [0]
3|   1 -> 1
4|
5|
6|
7|
8|
9|
```

FAIL (Type Error)

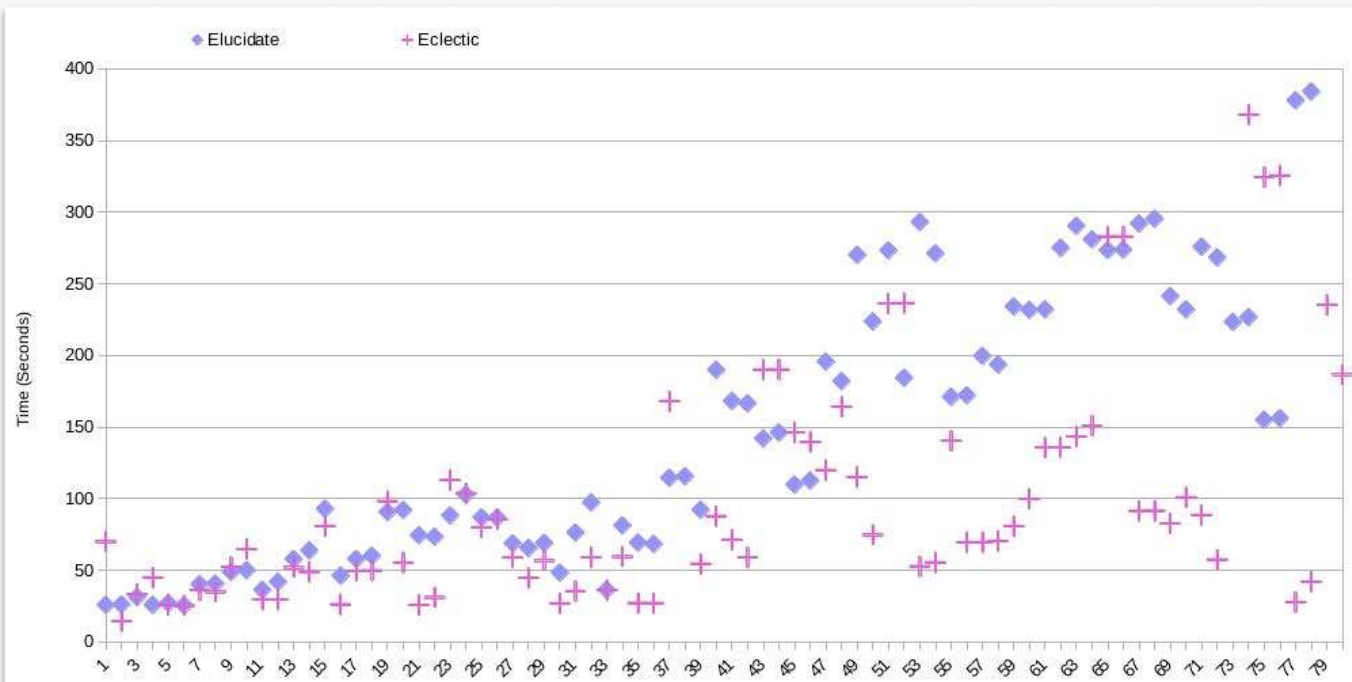
```
1|
2|
3|
4| plus :: Int -> Int -> Int
5| plus = (+)
6|
7|
8|
9|
```

PASS (Compiles)

Evaluation

- Evaluated with a scalability data-set based on Pandoc
 - 80 type errors, 2 placed in each of 40 chosen modules
 - Modules have between 32 to 2305 lines of code
- Comparison with our moiety debugger Elucidate
- Can we reduce the debugger run-time?

Reduce the run-time of the



Evaluation

- Testing the Agnostic ability of the debugger
 - 11 test programs in both Haskell and Ocaml
 - Blackboxes: GHC and Ocamlc
- Promising initial results show debugger is agnostic
- Correctly located 8 identical type errors in both languages

Future Work

- Heuristics to choose pre-processing or request
- An empirical study with real programmers
- More evaluation of the debuggers agonistic behavior

Thank You

- Shown an agnostic type error debugger using a Blackbox compiler, Delta Debugging, Moiety and Good-Omens Algorithms
 - Increased the average speed by **1.5 minutes**
 - Best **38 minutes** faster, worst **8 minutes** slower
 - Agnostic debugging a success needing more research